Design a Blackjack Game

Design a Hotel Reservation System

Design a Link-Shortener System

Design a Ride-Sharing Service

Design a Vending Machine

Design a Movie Ticket Service

Design a Stock Exchange System

Design a Tic-Tac-Toe Game

Design a Blackjack Game

Design a Shopping Cart System

Design an File System

Design a Search Autocomplete System

11
Design a Blackjack Game

In this chapter, we will discuss the object-oriented design of the Blackjack game (also called 21). Blackjack is a popular card game where the goal is to get cards that add up to 21 without going over, while also as close to possible, without going over. The game is a form of strategy, alleviating some of its chance and luck (the casino pun), making it a captivating and iconic casino game.

Let's gather the generic requirements through a typical interview-style conversation.

Blackjack Game

Requirements Gathering

Interviewer: Let's begin by gathering requirements.

Candidate: Should we consider a single-player or multi-player game?

Interviewer: Let's keep it simple with just one player against the house.

Candidate: Should we support standard Blackjack rules?

Interviewer: Yes, we'll stick to the classic Blackjack rules.

Candidate: Should the player bets, or should we assume a fixed bet?

Interviewer: Yes, the player should place bets, and we'll track the total bet. Don't worry about insurance or side bets.

Candidate: Do we need to handle ties, pushes, and blackjack payouts?

Interviewer: Absolutely! Ties and blackjack payouts are important.

Candidate: Should the dealer follow any specific rules for when to hit or stand?

Interviewer: Yes, the dealer must hit until the hand totals at least 17. Once they reach 17 or higher, they must stand.

Candidate: Do we need to persist the game, and how are players used?

Interviewer: Players place bets and receive payouts, but we don't need to persist data. Assume a player exists in their bet slot, or else we wait for one to join.

Requirements

As we discussed in the conversation, here are the key functional requirements we've identified:

- **Rules & Gameplay:** The Blackjack rules must be enforced, including the dealer's hit/stand logic.
- **Bets:** Players place bets, and the total bet is tracked.
- **Payouts:** Payouts must be calculated correctly, including for blackjack and ties.
- **Players:** Support one player at a time.

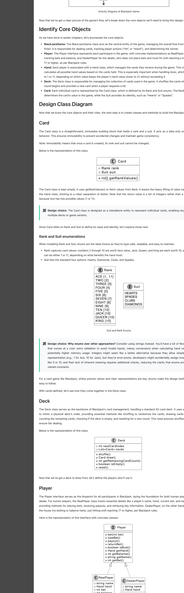
Non-functional requirements:

- **Extensibility:** The system should be easy to add new rules or features later.
- **Performance:** Game logic should be efficient and responsive.
- **Clarity:** The code should be clean, readable, and well-structured.

Activity Diagram

Understanding the flow of a game like Blackjack is crucial when designing its object-oriented structure, especially given the general rule of separating logic from data. This is where an activity diagram comes into play. We actively diagram (hence the name) the workflow of the game, capturing each action, decision, and transition in a clear, step-by-step manner.

In the context of Blackjack, this means visualizing everything from dealing cards and calculating scores, ensuring we account for all possible paths, such as a player busting or the dealer hitting until 17. Let's look at the activity diagram for Blackjack, which captures the process in detail:



Activity Diagram of Blackjack Game

Now that we've got a clear picture of the game's flow, let's break down the core objects we'll need to bring this design to life.

Identify the Objects

As we have seen in earlier chapters, let's enumerate the core objects.

- **Card:** represents a single card, including its suit and rank.
- **Deck:** represents the shuffling card deck, handling draw actions (`hit`) and determining the shoe.
- **Hand:** represents a collection of cards held by a dealer or player, calculating the total score and managing aces.
- **Player:** represents a human player, tracking their bankroll, managing their active hand, and processing bet adjustments.
- **Game:** represents the overall game, orchestrating the flow between players and the dealer.

The Card class is least simple, it uses getRankValue() to fetch values from data. It leaves the heavy lifting of value calculations to the Hand class, which knows how to handle aces and face cards. Note that this doesn't mean all A-J shapes create a hand, it just means it knows how to handle them.

```
Card
- Rank rank
- Suit suit
+ int getRankValue()
```

The Card class is least simple, it uses getRankValue() to fetch values from data. It leaves the heavy lifting of value calculations to the Hand class, which knows how to handle aces and face cards. Note that this doesn't mean all A-J shapes create a hand, it just means it knows how to handle them.

```
Deck
- List<Card> cards
+ void shuffle()
+ Card dealCard()
```

Full and Partial Decks

Design choice: **Why assume none other approaches?** Consider using strings instead. You'd have a lot of flexibility, but it introduces risk. Typos like "Hearts" vs. "Heart" can cause bugs, and you'll need extra validation. Instead, we'll use enums for rank and suit. This makes the code more robust, ensures the compiler helps catch bugs, and offers alternative because they allow cleaner method representations. (e.g., `rank.isFace()`). Yes, it's more work, but they're worth it. Enums also let us attach properties, like associating rank with its numerical value. This centralizes logic, ensuring consistency across the codebase. Overall, it's a better fit for this problem.

For a card game like Blackjack, where precise values and clear representations are key, enums make the design both robust and easy to follow.

With core definitions, let's see how they come together in the Hand class.

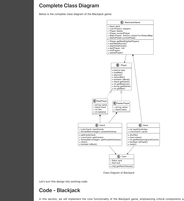
```
Hand
- List<Card> handCards
+ void addCard(Card pCard)
+ int getValue()
+ int getRankCount(Rank pRank)
+ boolean isBlackjack()
+ boolean isBust()
```

Now that we've got a deck to draw from, let's deal to the players and Dealer.

Player

The Player interface serves as the blueprint for all participants in Blackjack, laying the foundation for both human players and the dealer. It defines core actions like `getBet()` for managing wagers and `getHand()` to access the current cards. The interface also includes promising methods for placing bets, receiving payouts, and retrieving key information (`getPlayerId()`). On the other hand, dependent the house be nothing to balance hand, just hitting and reaching 17 is higher, as Blackjack rules.

Here's the representation of the interface with concrete classes:



Player Interface with concrete classes

Design choice: An interface-based design for Player is chosen to abstract common behaviors across human players and AI bots, promoting extensibility.

Now that our players are set, let's look at how they'll manage their cards with the Hand class.

Hand

The Hand class manages the cards for a player or dealer in Blackjack, keeping track of a list of cards and calculating all possible hand values. It's responsible for adding cards, checking for bust or blackjack conditions, and determining the best valid score. Key methods like `addCard()` and `getValue()` provide the core logic, ensuring the card list remains private and accessed only through a controlled interface.

Apparently, clear this hand for a new round, and determine if the hand is a pair (two cards of the same rank).

```
Hand
- List<Card> handCards
+ void addCard(Card pCard)
+ int getValue()
+ void clearHand()
+ boolean isPair()
+ int getRankCount(Rank pRank)
+ boolean isBlackjack()
+ boolean isBust()
```

Design choice: Keeping hand separate from Player keeps the design clean and modular by splitting responsibilities—Hand focuses solely on managing the cards and their values, while Player handles other game-specific details like bets and balance.

With hands managing cards and totals, let's see how the BlackjackGame class ties it all together.

BlackjackGame

The BlackjackGame class is the central entity in our Blackjack design, orchestrating the action from the initial deal to dealing the end of each round. It oversees the game's decks, the players, and the dealer, handling card distribution, tracking turns, and ultimately determining the outcome for each participant. This class is the heart of the system, ensuring that everything from bets to hits and stands flows smoothly and adheres to the game's rules.

Here's the representation of this class.

```
BlackjackGame
- Deck pDeck
- List<Player> pPlayers
- int numPlayers
- int numDecks
- Map<Player, Boolean> blackjackMap
- int currentPlayerIndex
- int currentHandIndexForPlayer
+ void startNewRound()
+ void dealInitialCards()
+ void processPlayerTurn(Player pPlayer)
+ void processDealerTurn()
+ void calculatePayouts()
+ void resetGame()
+ void addPlayer(Player pPlayer)
+ void removePlayer(int pPlayerId)
+ int getNumPlayers()
+ int getNumDecks()
+ void setNumDecks(int pNumDecks)
```

Now that we've detailed our key classes, let's put them together in a complete class diagram to see the full picture.

Complete Class Diagram

Below is the complete class diagram of the Blackjack game:

Class Diagram of Blackjack

Let's turn this design into working code.

Code - Blackjack

In this section, we will implement the core functionality of the Blackjack game, emphasizing critical components such as deck management, hand evaluation, bankroll tracking, player turns, and payouts. We'll skip the specifics and shorter demonstration logic for brevity.

Card

The Card class is a straightforward representation of a playing card, combining a Rank and a Suit. It's designed to be immutable, ensuring that once a card is created, its properties remain unchanged. The getRankValue() method retrieves the card's possible values from Rank, e.g., 11 for an Ace.

Below is the code implementation of this class:

```
class Card {
public final Rank rank;
public final Suit suit;

public Card(Rank rank, Suit suit) {
    this.rank = rank;
    this.suit = suit;
}

public int[] getRankValues() {
    return rank.getRankValues();
}
}
```

Next is a enum that defines card values. Aces can be 1 or 11, traditional cards retain their face value, and face cards (Jack, Queen, King) all equal 10. It stores these values in an array and provides them through getRankValues().

```
enum Rank {
    ACE(1, 11), TWO(2),
    THREE(3), FOUR(4),
    FIVE(5), SIX(6),
    SEVEN(7), EIGHT(8),
    NINE(9), TEN(10),
    JACK(10), QUEEN(10),
    KING(10);

    private int[] values;

    Rank(int... values) {
        this.values = values;
    }

    public int[] getRankValues() {
        return values;
    }
}
```

Suits are a simple enum with four options: Hearts, Spades, Clubs, and Diamonds. It acts purely as a label, giving each card its suit without logic.

```
enum Suit {
    HEARTS,
    SPADES,
    CLUBS,
    DIAMONDS
}
```

With cards ready, let's look how the deck manages them.

Deck

The Deck class represents a full deck of playing cards and provides essential operations such as shuffling, drawing, and resetting. It maintains a dynamic collection of Card objects and ensures that cards are dealt in the correct sequence.

Here's the implementation of this class:

```
public class Deck {
    private int numDecks;
    private List<Card> cards;

    // Constructor initializes the deck
    public Deck(int numDecks) {
        this.numDecks = numDecks;
        initializeCards();
    }

    // Initializes the deck with all cards
    private void initializeCards() {
        cards = new ArrayList<>();
        for (int deck = 0; deck < numDecks; deck++) {
            for (Rank rank : Rank.values()) {
                for (Suit suit : Suit.values()) {
                    cards.add(new Card(rank, suit));
                }
            }
        }
        numCardsLeft = 52 * numDecks;
    }

    // Shuffles the deck using Collections.shuffle()
    public void shuffle() {
        Collections.shuffle(cards, new Random(System.currentTimeMillis));
    }

    // Draws the next card from the deck
    public Card dealCard() {
        if (numCardsLeft == 0) {
            System.out.println("No more cards in the deck.");
            return null;
        }
        int lastIndex = cards.get(numCardsLeft);
        numCardsLeft--;
        return lastIndex;
    }

    // Returns the number of remaining cards in the deck
    public int getRemainingCardsCount() {
        return cards.size() - numCardsLeft;
    }

    // Checks if the deck is empty
    public boolean isEmpty() {
        return getRemainingCardsCount() == 0;
    }

    // Resets the deck to a new shuffled state.
    public void reset() {
        numCardsLeft = 0;
        shuffle();
    }
    // Other methods are omitted for brevity
}
```

The Deck is initialized by calling each Rank with each Suit, creating a complete set of 52 cards. These cards are stored in a private List<Card>, ensuring controlled access to the deck's contents.

implementation choice: Instead of creating a new shuffled deck for each round, the deck tracks how many cards remain and reshuffles when needed. This avoids the costly overhead of constantly creating new objects, which saves memory and improves performance. This approach is ideal for simulations or long-running games, where efficiency matters.

Now let's look at a game handy function that deals the initial cards and checks for blackjack.

Hand

The Hand class is responsible for managing a player's cards and calculating possible hand totals, particularly when dealing with aces. Aces can count as either 1 or 11, so this class must track all valid scores and determine the best one to use without going over 21.

The core logic is in `getValue()`, which calculates all possible hand totals by summing card values and adjusting for aces if needed. The method returns the highest valid total or the lowest one if all options bust. This ensures the player always gets the most favorable hand without exceeding the limit.

```
public class Hand {
    private List<Card> handCards;

    // Constructor
    public Hand(List<Card> pCards) {
        this.handCards = pCards;
    }

    // Calculates the best valid hand value.
    // Prioritizes hands under 21.
    public int getValue() {
        int sum = 0;
        int aceCount = 0;
        for (Card card : handCards) {
            int[] values = card.getRankValues();
            sum += values[0];
            if (values.length > 1 && values[0] == 1) {
            // Ace has two values, 1 and 11.
            aceCount++;
            }
        }
        while (aceCount > 0 && sum + 10 <= 21) {
            sum += 10;
            aceCount--;
        }
        return sum;
    }

    // Checks if the hand is a pair.
    public boolean isPair() {
        return handCards.size() == 2 &&
            handCards.get(0).getRank() == handCards.get(1).getRank();
    }

    // Other methods are omitted for brevity
}
```

implementation choice: Instead of creating a new shuffled deck for each round, the deck tracks how many cards remain and reshuffles when needed. This avoids the costly overhead of constantly creating new objects, which saves memory and improves performance. This approach is ideal for simulations or long-running games, where efficiency matters.

Now that we've got cards flowing from the deck, let's see how they land in a player's hand

```
    int getBet();
}
```

The RealPlayer class models a human player who engages in betting and managing their funds. It implements the Player interface, ensuring that bet handling is managed separately from the core game logic. When placing a bet, the bet() method ensures that the bet does not exceed the player's available balance before deducting the amount.

To maintain a clear separation of concerns, RealPlayer does not handle card evaluation directly. Instead, it delegates all card-related operations to the Hand class. This ensures that the player's bet handling and card logic remain separate, making the class more modular and maintainable.

```
public class RealPlayer implements Player {
    private final int startBalance;
    private final Hand hand;
    private final int bet;
    private final int balance;

    public RealPlayer(String name, int startBalance) {
        this.name = name;
        this.hand = new Hand();
        this.bet = 0;
        this.balance = startBalance;
    }

    // Places a bet for the player
    @Override
    public void bet(int bet) {
        if (bet > balance) {
            throw new IllegalArgumentException("Bet is greater than balance");
        }
        this.bet = bet;
        this.balance -= bet;
    }

    // Handles the player losing a bet
    @Override
    public void loseBet() {
        this.bet = 0;
    }

    // Handles returning the player's bet
    @Override
    public void returnBet() {
        this.balance += bet;
        this.bet = 0;
    }

    // Handles the player winning a payout
    @Override
    public void payout(int p) {
        this.balance += bet * 2; // Return bet plus equal amount
        this.bet = 0;
    }

    // getter methods are omitted for brevity
}
```

The DealerPlayer class represents the house and follows predefined rules that differ from those of human players. Since the dealer does not participate in betting, the bet-handling methods, bet(), and loseBet() are implemented as empty functions, as they are never invoked in gameplay. Similarly, getBalance() and getBet() always return 0, reflecting the fact that the dealer does not manage a balance or place bets.

```
public class DealerPlayer implements Player {
    private final String name = "Dealer";
    private final Hand hand;

    public DealerPlayer() {
        this.hand = new Hand();
    }

    // Bet handling methods for Dealer (bet, loseBet, returnBet) are implemented as empty functions.

    @Override
    public void payout(int p) {
        // Dealer does not get a payout, so this method only prints the winning hand
    }
    // getter methods are omitted for brevity
}
```

With players and their hands set, let's orchestrate the whole game in BlackjackGame.

BlackjackGame

The BlackjackGame class serves as the central orchestrator of the game, handling players, the dealer, the deck, turn management, and enforcing the game rules. It ensures that the game follows a structured flow, with each player's actions and the final outcome being resolved according to the rules of Blackjack.

Below is the implementation of this class.

```
public class BlackjackGame {
    private final Deck deck = new Deck();
    private final List<Player> players = new ArrayList<>();
    protected final Player dealer = new DealerPlayer();
    private Player currentPlayer = null;

    // Tracks the current status of each player's turn (e.g., HIT or STAND)
    Map<Player, Action> playerTurnStatusMap = new HashMap<>();
    GamePhase currentPhase = GamePhase.START;

    public BlackjackGame(List<Player> p) {
        for (Player player : p) {
            if (player == null) throw new IllegalArgumentException();
            this.players.add(player);
            this.playerTurnStatusMap.put(player, null);
        }
        this.playerTurnStatusMap.put(dealer, null);
        deck.shuffle(); // Shuffle the deck when game starts
    }

    // Determines the next player who can take an action (i.e., has not stood or bust). If the
    // current player is the dealer, it triggers the dealer's turn.
    // Returns null if no players remain.
    // If current player hasn't stood or bust, they can continue their turn
    public CurrentPlayer * nd();
        if (currentPlayer != null && !playerTurnStatusMap.getCurrentPlayer())
            && !currentPlayer.isBust())
            return currentPlayer;

        // Find the first player who hasn't stood or bust
        int currentPlayerIdx = 0;
        for (Player player : players) {
            if (!Action.STAND.equalsPlayerTurnStatusMap.get(player) && !player.isBust()) {
                currentPlayer = player;
                return currentPlayer;
            }
            currentPlayerIdx++;
        }

        // Also, find the next player after the current one who hasn't stood or bust
        int currentPlayerIdx = players.indexOf(currentPlayer);
        for (int i = currentPlayerIdx + 1; i < players.size(); i++) {
            Player player = players.get(i);
            if (!Action.STAND.equalsPlayerTurnStatusMap.get(player) && !player.isBust()) {
                currentPlayer = player;
                return currentPlayer;
            }
        }

        currentPlayer = player;
        return currentPlayer;
    }

    // If no players are left to act, return null
    return null;
}

    protected void dealerTurn() {
        // Dealer hits if below 17
        int dealerHandValue = getPossibleValues().last() - 17;
        int numDraw = deck.draw();
        dealer.getHand().addCard(numDraw);

        playerTurnStatusMap.put(dealer, Action.STAND);
        checkGameEndCondition();
    }

    public void startRoundLoop() {
        while (true) {
            Player CPlayer = playerTurnStatusMap.keySet();
            if (CPlayer.getHand().size(), // Clear dealer's hand
            dealer.getHand().clear();
            playerTurnStatusMap.replaceAll((p, v) -> null);
            currentPlayer = null; // Reset current player
            currentPhase = GamePhase.STARTED;

            int numBetsToCollect = 0;
            if (!GamePhase.BET_PLACED.equals(currentPhase)) {
                throw new IllegalStateException("All players must bet before dealing");
            }

            // Deal first card to each real player in order
            for (Player player : players) {
                player.getHand().addCard(deck.draw());
            }

            // Deal first card to dealer
            int numDraw = deck.draw();
            dealer.getHand().addCard(deck.draw());

            // Deal second card to each real player in order
            for (Player player : players) {
                player.getHand().addCard(deck.draw());
            }

            // Deal second card to dealer
            numDraw = deck.draw();
            dealer.getHand().addCard(numDraw);

            currentPhase = GamePhase.INITIAL_CARD_DRAWN;
        }
    }

    public void bet(Player player, int bet) {
        if (GamePhase.STARTED.equals(currentPhase)) {
            throw new IllegalStateException("Bets must be placed at the start of the round");
        }
        player.bet(bet);
        // Transition to BET_PLACED once all players have bet
        if (players.stream()
                .allMatch(p -> p.getBet() > 0
                    && p.getBet() == GamePhase.BET_PLACED;
        }
    }

    public void hit(Player player) {
        if (Action.STAND.equalsPlayerTurnStatusMap.get(player)) {
            throw new IllegalStateException("Player has already stood");
        }
        if (player.isBust()) {
            throw new IllegalStateException("Player is already bust");
        }

        Card drawnCard = deck.draw();
        player.getHand().addCard(drawnCard);
        playerTurnStatusMap.put(player, Action.HIT);
    }

    public void stand(Player player) {
        if (Action.STAND.equalsPlayerTurnStatusMap.get(player)) {
            throw new IllegalStateException("Player has stood");
        }
        if (player.isBust()) {
            throw new IllegalStateException("Player is already bust");
        }
        playerTurnStatusMap.put(player, Action.STAND);
    }

    // Checks if the game has ended (all players done), then resolves bets by comparing each
    // player's hand to the dealer's.
    private void checkGameEndCondition() {
        boolean allPlayersDone = true;
            players.stream()
                .allMatch(p ->
            if (!allPlayersDone) { // Action.STAND.equalsPlayerTurnStatusMap.get(p) || p.isBust();
                return;
            }

            int dealerValue = dealer.getHand().getPossibleValues().last();
            boolean dealerBust = dealer.isBust();

            for (Player player : players) {
                if (player.isBust()) {
                    player.loseBet();
                } else {
                    int playerValue = player.getHand().getPossibleValues().last();
                    if (dealerBust || playerValue > dealerValue) {
                        player.payout(2);
                    } else if (playerValue == dealerValue) {
                        player.returnBet();
                    } else {
                        player.loseBet();
                    }
                }
                currentPhase = GamePhase.END;
            }

    // getter methods are omitted for brevity
}
```

- Each round begins with startRoundLoop(), which resets the deck, ensuring that players start with a clean slate.
- Once the round starts, players place their bets using the bet() method, then startRoundLoop() distributes two cards per player, including the dealer. This guarantees a fair and structured start before betting and player actions begin.
- The game progresses through player turns using getNextCurrentPlayer(), which selects the next active player who has not yet stood or busted. Once all players have finished their turns, control shifts to the dealer via dealerTurn().

The dealer follows strict, predefined rules:

- The dealer must hit until their total reaches at least 17.
- Once at 17 or higher, the dealer must stand.

These rules ensure consistent and fair gameplay, mirroring real-world Blackjack dealer behavior.

- hit() Draws a card and updates the player's hand. If the player exceeds 21, they are marked as bust, eliminating them from further action.
- stand() Locks in the player's hand, preventing further draws.

The checkGameEndCondition() method finalizes the round once all players have stood or busted. It compares each player's hand individually against the dealer's:

- If the dealer busts (hand exceeds 21), non-bust players win and receive a payout.
- For non-bust players, if their hand value exceeds the dealer's (but is ≤ 21), they win and are paid out. If it's lower, they lose their bet. In the case of a tie, return the bet to the player.

Now that we've built a working game, let's explore how we can make it more flexible for future changes

Deep Dive Topic

At this point, you've nailed the core requirements of the Blackjack game. This section will dive deeper into a potential extension to make the design more adaptable.

Decoupling player and dealer decision logic

In the current design, BlackjackGame directly controls player actions, calling hit() or stand() based on predefined conditions in the game logic. Similarly, the dealer's "hit until 17" rule is hardcoded into dealerTurn(). This approach tightly couples decision-making with the game class, leaving little room for custom moves. Any rule modifications, such as allowing a cautious player to stand at 15 or adjusting the dealer's hit threshold, requires modifying BlackJackGame, leading to complex maintenance and potential bugs.

To resolve this, we introduce a decision-making abstraction that shifts control to individual players. This keeps BlackJackGame focused on coordinating turns, while each player determines their moves independently.

Step 1: Define a decision-making interface

To give each player control over their moves, we create a PlayerDecisionLogic interface with one method, decideAction(Hand). Real picks "Hit" or "Stand" based on the hand:

```
interface PlayerDecisionLogic {
    // Decides the next action for a player based on their hand
    Action decideAction(Hand hand);
}
```

Step 2: Tailor decisions for humans and dealers

With the interface in place, we implement two concrete decision classes:

- RealPlayerDecisionLogic: This captures a human player's approach, letting the user hitting if the hand's below 18.
- DealerDecisionLogic: This locks in the dealer's rule, hit if under 17.

Here's this code:

```
public class RealPlayerDecisionLogic implements PlayerDecisionLogic {
    @Override
    public Action decideAction(Hand hand) {
        return hand.getPossibleValues().last() < 18 ? Action.HIT : Action.STAND;
    }
}

public class DealerDecisionLogic implements PlayerDecisionLogic {
    @Override
    public Action decideAction(Hand hand) {
        return hand.getPossibleValues().last() < 17 ? Action.HIT : Action.STAND;
    }
}
```

Step 3: Integrate decisions into players

We update the Player interface with a getDecisionLogic() method, letting each player define its decision style. RealPlayer defaults to RealPlayerDecisionLogic, while DealerPlayer uses DealerDecisionLogic:

```
public interface Player {
    // Returns the decision logic for the player
    PlayerDecisionLogic getDecisionLogic(); // ... other methods ...
}

public class RealPlayer implements Player {
    private final PlayerDecisionLogic decisionLogic;

public RealPlayer(String name, int startBalance) {
    this.name = name;
    this.hand = new Hand();
    this.bet = 0;
    this.balance = startBalance;
    this.decisionLogic = new RealPlayerDecisionLogic();
}

    // Returns the decision logic for the player
    @Override
    public PlayerDecisionLogic getDecisionLogic() {
        return decisionLogic;
    }
    // ... other methods ...

public class DealerPlayer implements Player {
    private final PlayerDecisionLogic decisionLogic;

    public DealerPlayer() {
        this.hand = new Hand();
        this.decisionLogic = new DealerDecisionLogic();
    }

    // Returns the decision logic for the dealer
    @Override
    public PlayerDecisionLogic getDecisionLogic() {
        return decisionLogic;
    }
    // ... other methods ...
}
```

Step 4: Adjust the game flow

In this step, we refactor the BlackJackGame code to use decision logic for both players and the dealer, streamlining the turn sequence. We introduce the performPlayerAction() method, which queries each player's decision logic (RealPlayerDecisionLogic or DealerDecisionLogic) to decide whether to hit or stand. This replaces the hardcoded dealerTurn() method, integrating the dealer into the same flow as the players.

We also add the playerTurn() method to coordinate turns, relying on getNextCurrentPlayer() to determine who acts next.

Here's the updated code:

```
public class BlackJackGame {
    // ... fields unchanged ...

    // Find the next player who can take an action
    public Player getNextCurrentPlayer() {
        // No current player: find first eligible player from the start
        if (currentPlayer == null) {
            for (Player player : players) {
                if (!Action.STAND.equalsPlayerTurnStatusMap.get(player) && !player.isBust()) {
                    return currentPlayer;
                }
            }
            // Instead of calling dealerTurn(), check if the dealer can act
            if (!Action.STAND.equalsPlayerTurnStatusMap.get(dealer)) {
                currentPlayer = dealer;
                return dealer;
            }
            return null;
        } else {
            int currentIndex = players.indexOf(currentPlayer);
            for (int i = currentIndex + 1; i < players.size(); i++) {
                Player player = players.get(i);
                if (!Action.STAND.equalsPlayerTurnStatusMap.get(player) && !player.isBust()) {
                    currentPlayer = player;
                    return currentPlayer;
                }
            }
            // If all players are done, check if the dealer can act
            if (!currentPlayer != dealer && !Action.STAND.equalsPlayerTurnStatusMap.get(dealer)) {
                currentPlayer = dealer;
                return dealer;
            }
            return null; // All turns are complete, including the dealer's
        }
        // Executes the next turn by acting for the next player or dealer.
        public void playNextTurn() {
            Player nextPlayer = getNextCurrentPlayer();
            if (nextPlayer != null) {
                performPlayerAction(nextPlayer);
            }
        }

        // Performs the action decided by the player's decision logic (hit or stand).
        private void performPlayerAction(Player player) {
            Action action = player.getDecisionLogic().decideAction(player.getHand());
            if (action == Action.HIT) {
                hit(player);
            } else if (action == Action.STAND) {
                stand(player);
            }
        }
        // ... other methods unchanged, dealerTurn() removed ...
    }
}
```

The performPlayerAction() method now handles the full turn sequence. It returns the next player who hasn't stood or busted, then the dealer once all players are done, and finally null when the dealer has stood, ending the turns. While dealerTurn() is removed, its logic lives on in DealerDecisionLogic, ensuring the dealer hits until 17 or higher.

This design shifts decision-making to the players, letting BlackJackGame coordinate turns while each player's logic lives separately in a tweak of the Strategy Pattern, you might notice this follows it, defining a set of decision rules that can swap in and out. Here:

- PlayerDecisionLogic sets the decision contract.
- RealPlayerDecisionLogic and DealerDecisionLogic are the specific behaviors.
- BlackJackGame uses them without needing to handle decisions internally.

Note: To learn more about the Strategy Pattern and its common use cases, refer to the **Factory List** chapter.

Wrap Up

In this chapter, we have built a solid Blackjack game from the ground up, with the key takeaway being how we structured responsibilities across Card, Deck, Hand, Player, and BlackJackGame into a clear, well-organized design. Each piece does its job, for instance, Card holds the essentials, Deck shuffles and deals, Hand tracks totals, and BlackJackGame runs the show. This approach keeps things neat, easy to follow, and scalable.

We also took things further by decoupling decision-making with PlayerDecisionLogic, making it easy to swap strategies for players and the dealer. If you're familiar with the Strategy pattern, you'll recognize how this is applied here, giving the game flexibility without rewriting.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Mark as Complete

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.